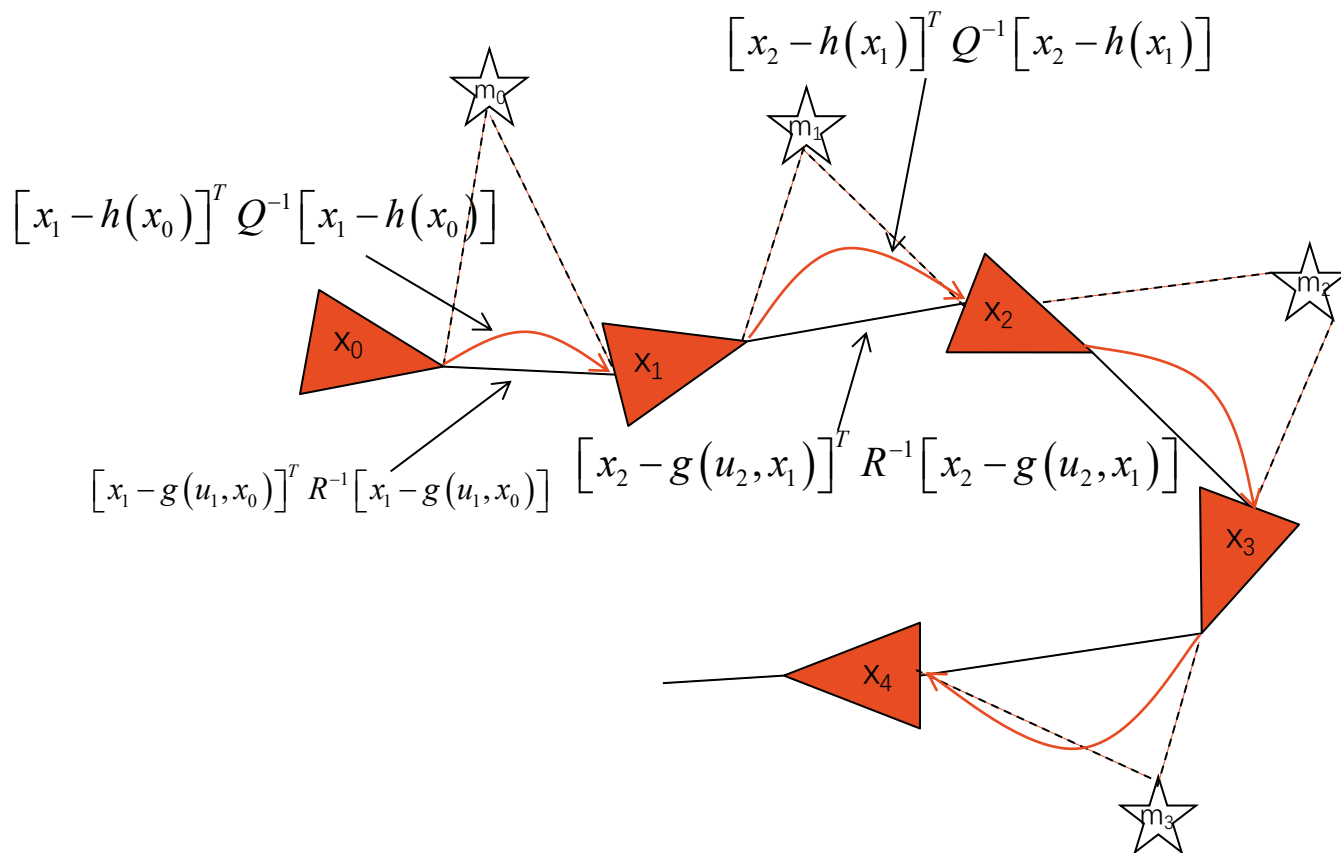


回环检测与相机重定位 (Loop Closure and Camera Relocalization)



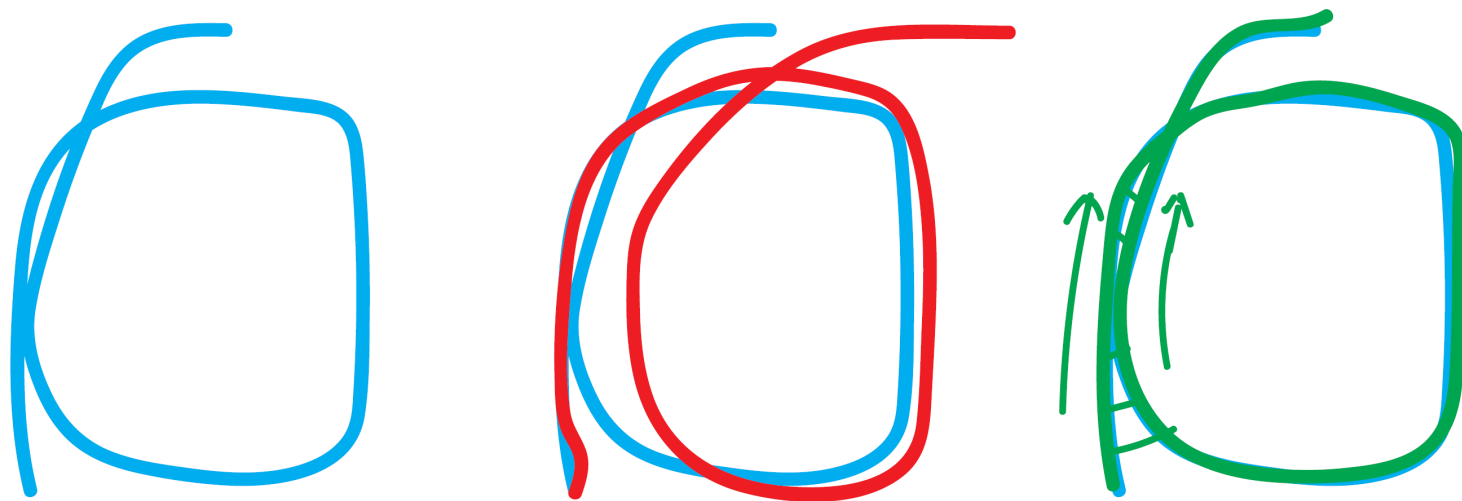
仅基于前端的位姿计算的过程



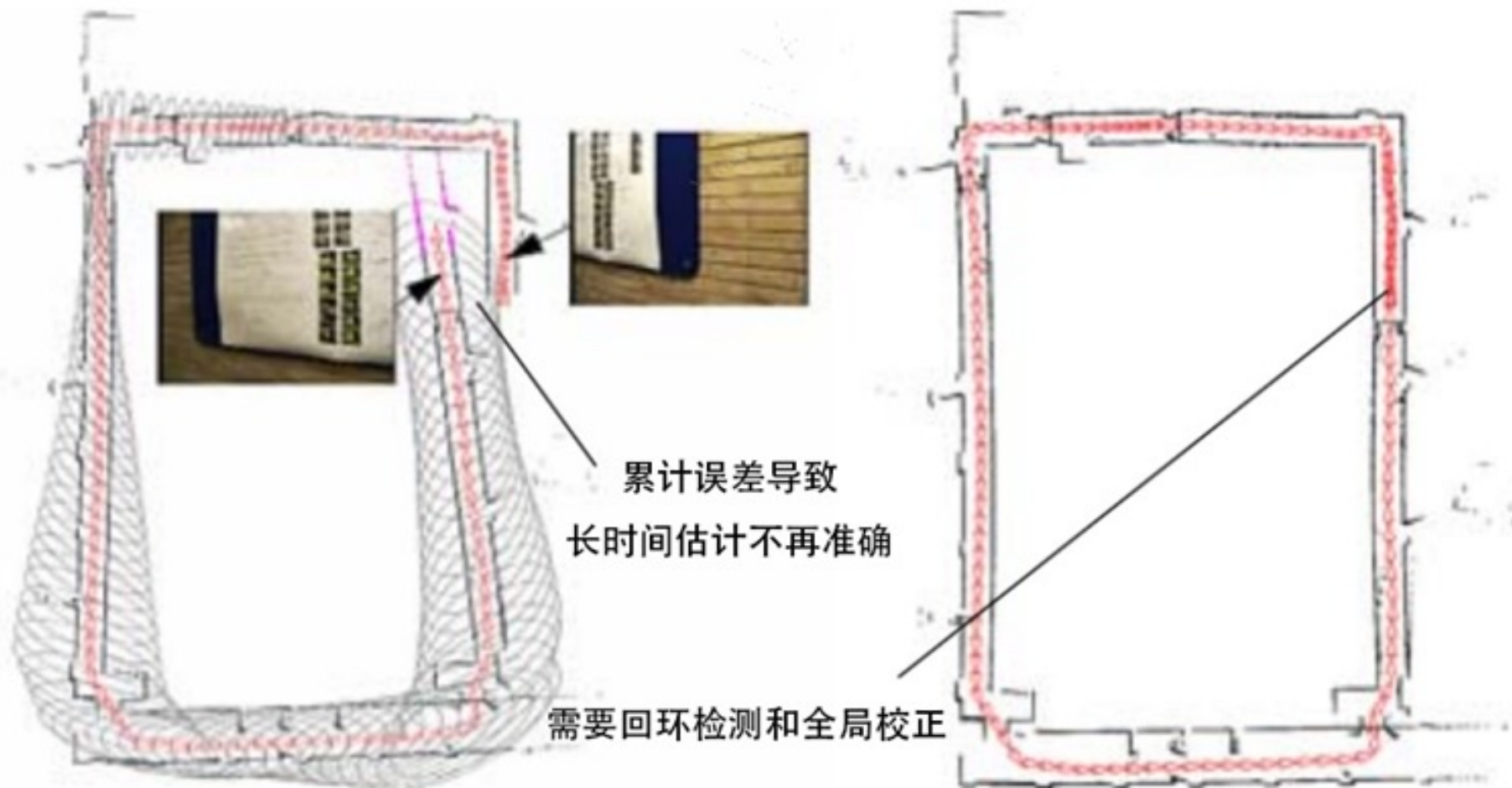
$$J_{GraphSlam} = x_0^T \Omega_0 x_0 + \sum_t [x_t - g(u_t, x_{t-1})]^T R^{-1} [x_t - g(u_t, x_{t-1})] + \sum_t [x_t - h(x_{t-1})]^T Q^{-1} [x_t - h(x_{t-1})]$$

回环检测的意义

- 前端给出的只是局部的位姿间约束，局部观测会产生累积误差
- 误差会被累积起来，使得后端优化的结果慢慢地趋向不准确
- 回环检测能给出**时隔更加久远**的约束，有助于产生全局一致的结果



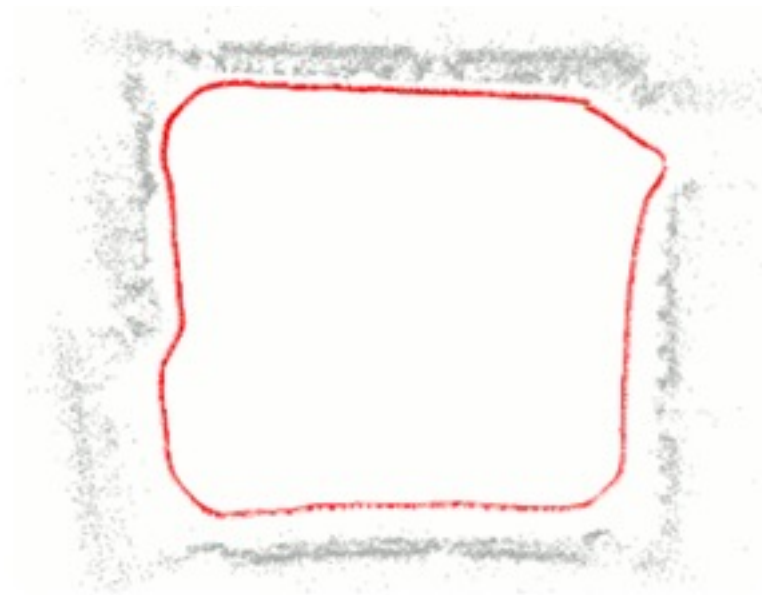
用回环检测和不用回环检测的差异



用回环检测和不用回环检测的差异

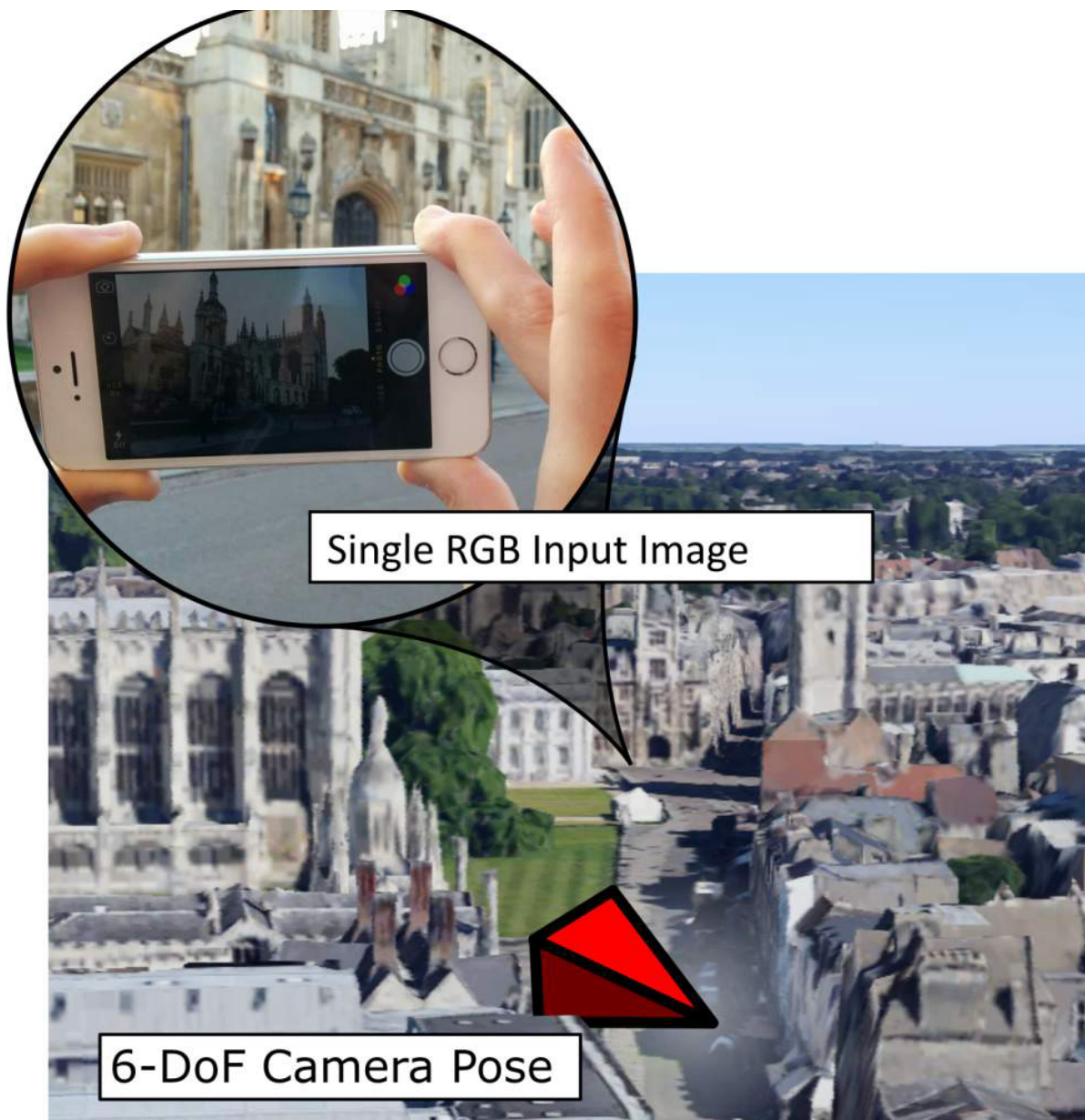


不用回环检测

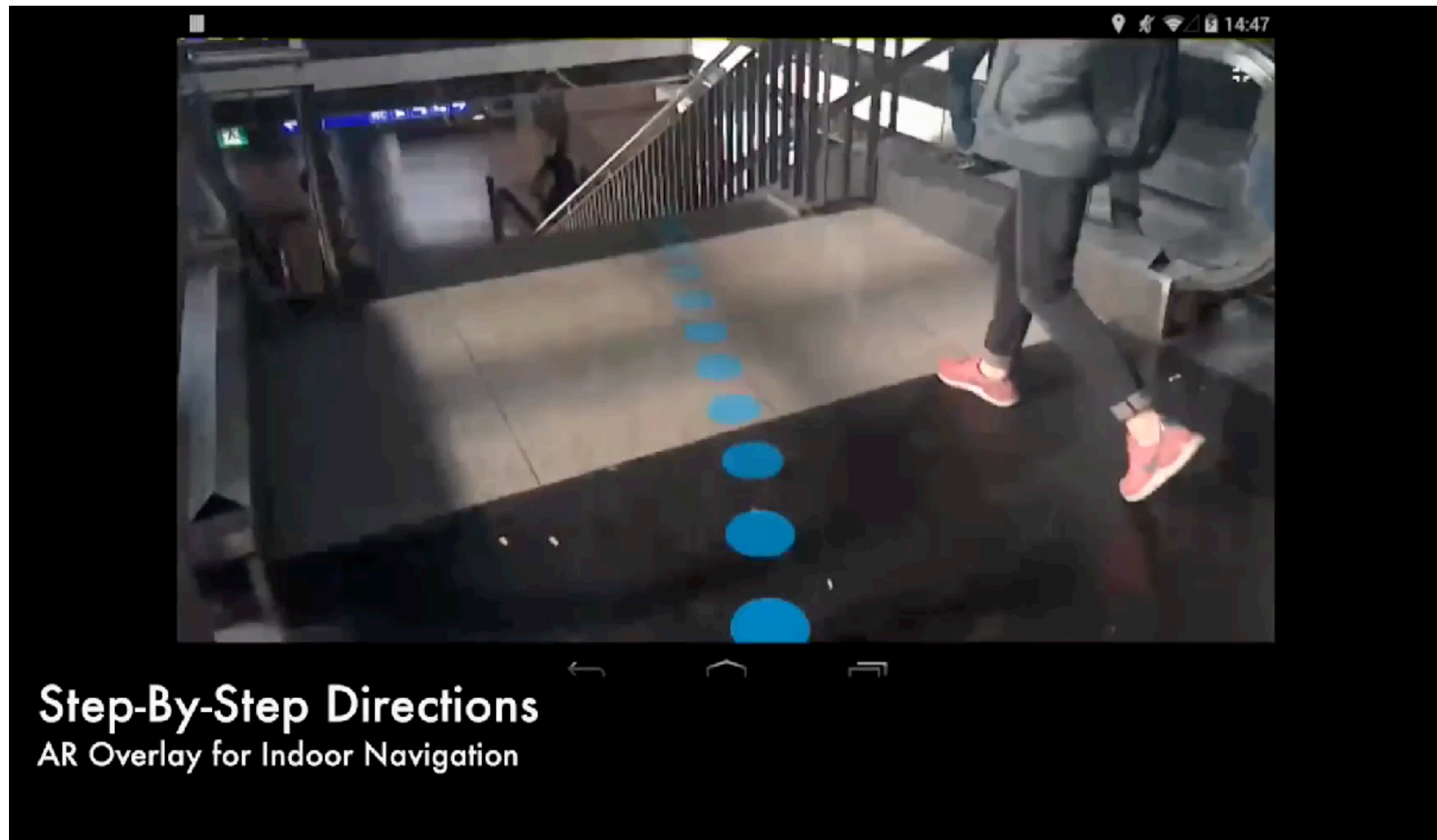


用回环检测

视觉重定位的意义

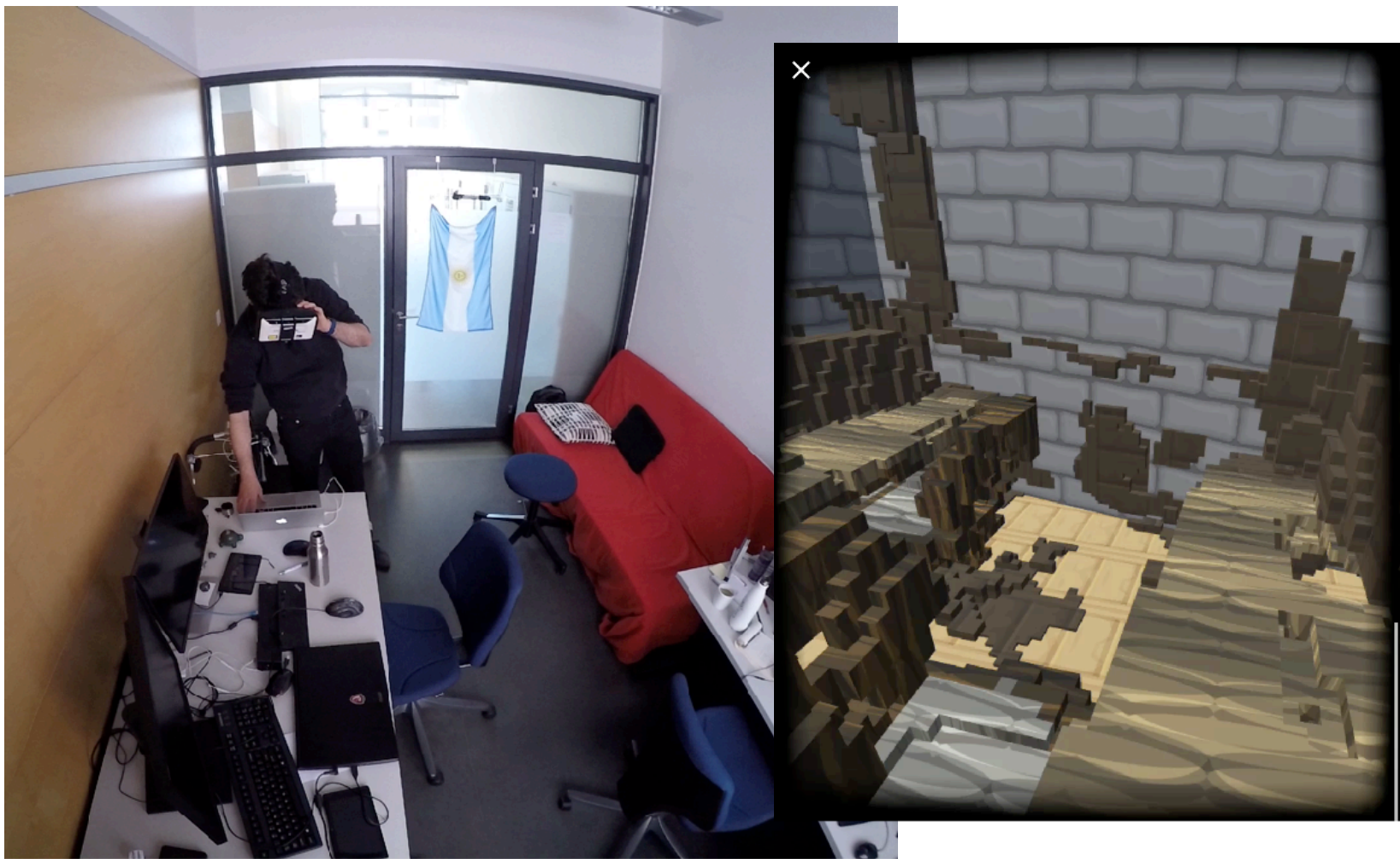


视觉重定位的意义



在AR应用中，基于视觉重定位结果进行导航

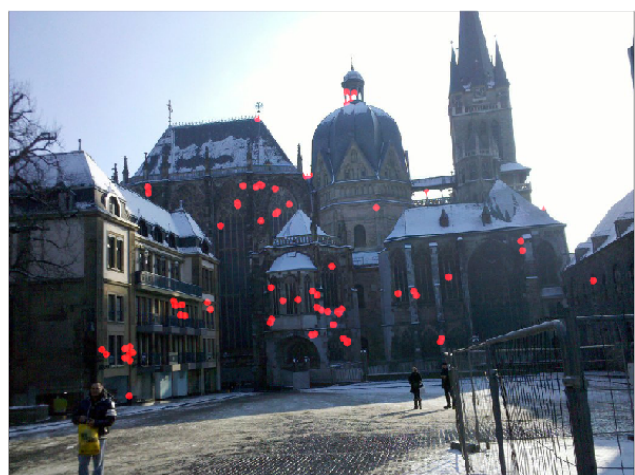
视觉重定位的意义



在交互式VR应用中，基于视觉重定位结果提供准确的位置和姿态信息

闭环检测和视觉重定位的主要方法

基于特征点匹配的方法

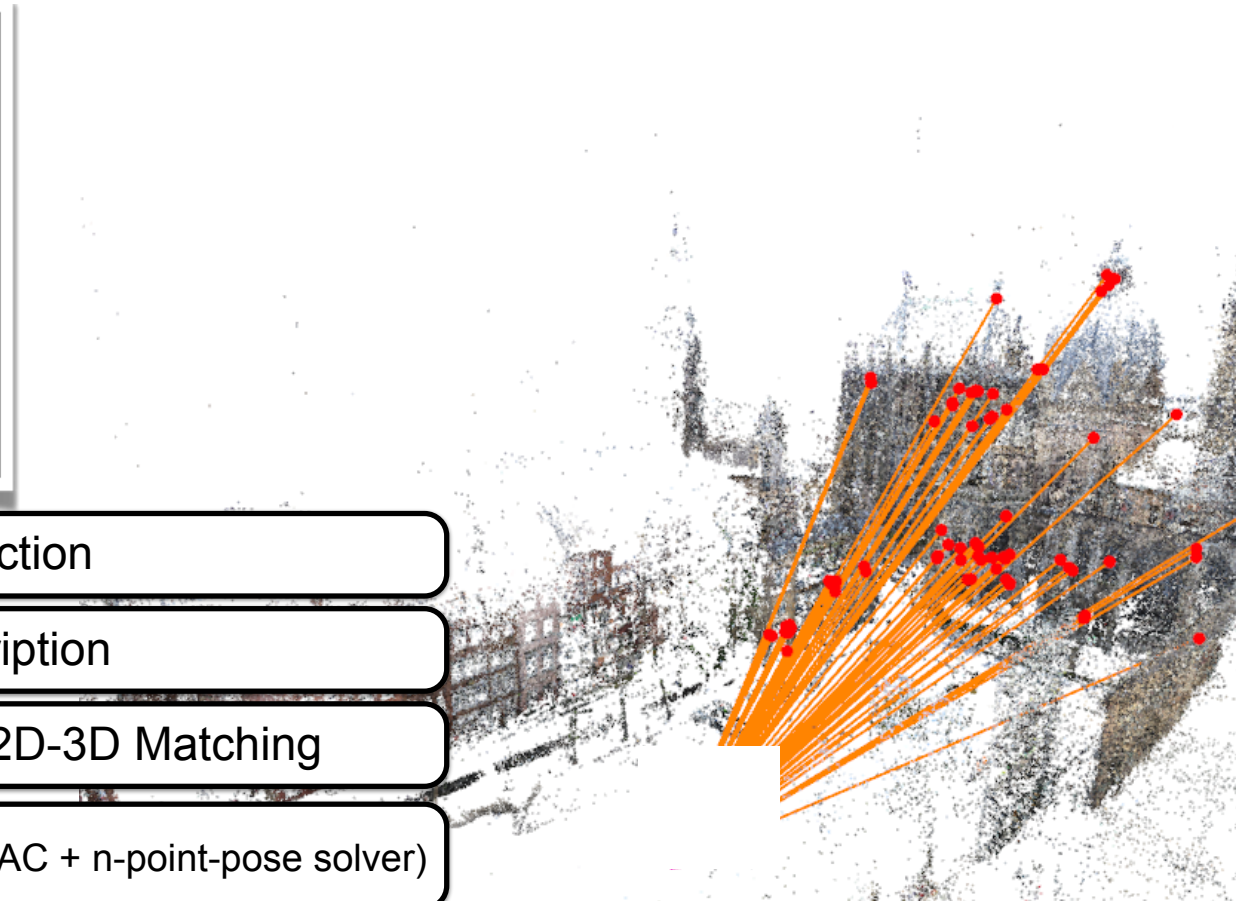


Feature Detection

Feature Description

Descriptor Matching for 2D-3D Matching

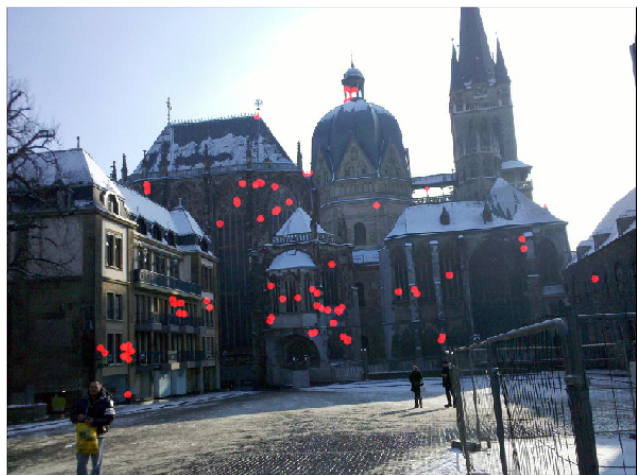
Estimate Camera Pose (RANSAC + n-point-pose solver)



基于词袋模型等，基于查询图像的特征点寻找匹配关键帧

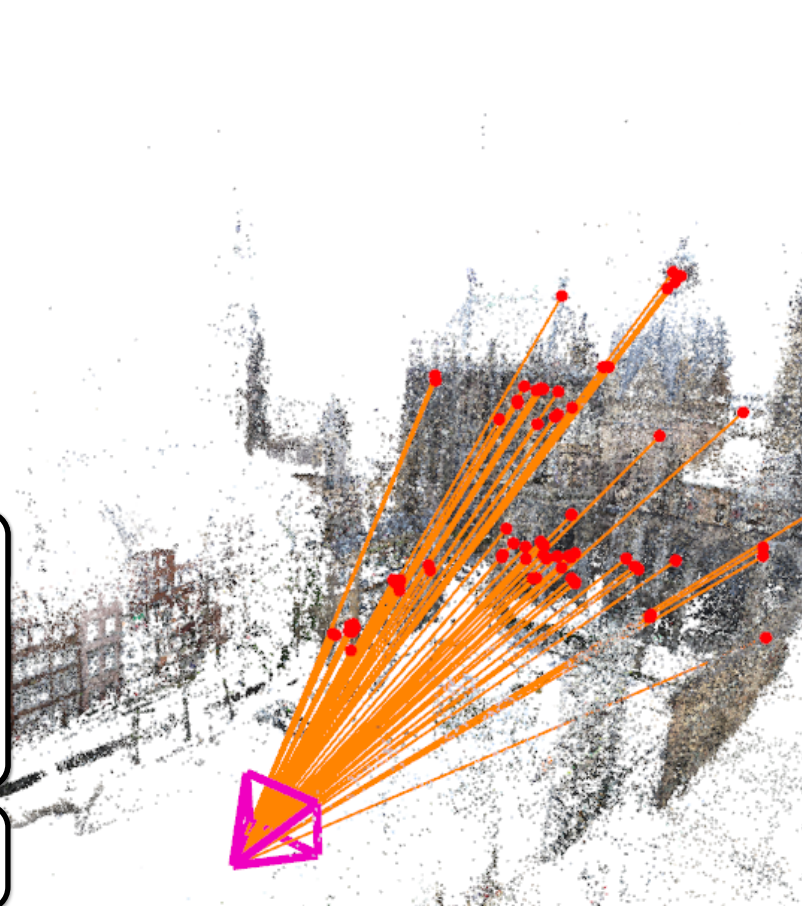
闭环检测和视觉重定位的主要方法

基于机器学习的场景匹配的方法



Predict 2D-3D Matches from Patches
(CNN, Random Forest, ...)

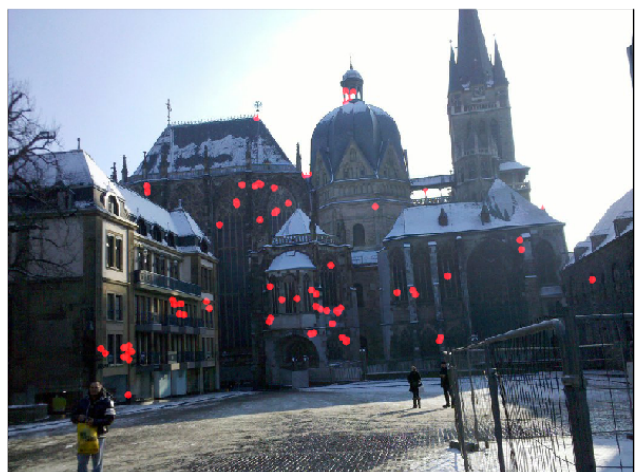
Estimate Camera Pose (RANSAC + n-point-pose solver)



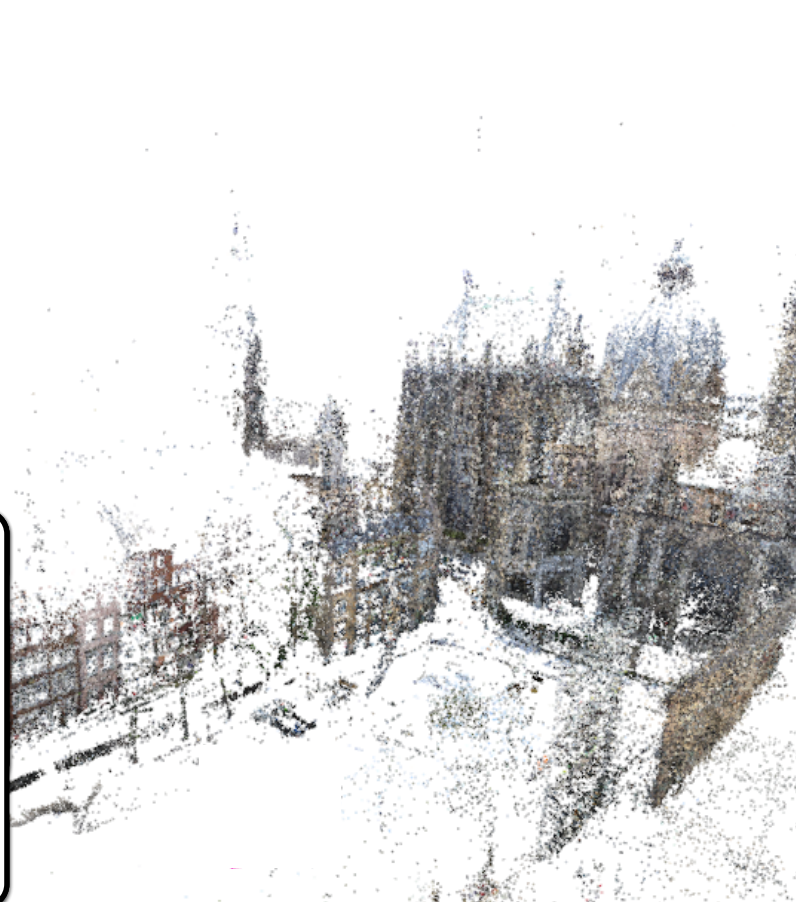
基于训练的机器学习模型，预测匹配关键帧

闭环检测和视觉重定位的主要方法

基于深度学习直接回归相机姿态的方法



Predict 6DOF Camera Pose from Image
(CNN)



基于训练的深度学习模型，直接回归相机的6DoF位姿

视觉重定位介绍

图像检索方法

从图像数据库中找到可能是回环的图像

代表性方法

DBoW

VLAD

NetVLAD

图像匹配方法

计算查询图像 S 与可能回环图像 T 之间的特征点匹配

代表性方法

暴力匹配, KNN

SuperGlue

Loftr

相对位姿计算方法

计算两张图像之间的相对位姿关系 T_{ST}

代表性方法

对极几何

RANSAC

PoseNet

视觉重定位的相关Tutorial

ICCV 2021 Tutorial, Large-Scale Visual Localization <https://sites.google.com/view/lsvpr2021/home>

- ICCV 2019 Tutorial - Large-Scale Visual Place Recognition and Image-Based Localization
- ECCV 2018 Tutorial - Visual Localization: Feature-based vs. Learned Approaches
- CVPR 2017 Tutorial - Large-Scale Visual Place Recognition and Image-Based Localization
- CVPR 2015 Tutorial - Large-Scale Visual Place Recognition and Image-Based Localization
- CVPR 2014 Tutorial - Large-Scale Visual Place Recognition and Image-Based Localization



基于特征匹配的回环检测方法

王永才

基于特征匹配的回环检测方法

1. 最简单的方式就是对任意两张图像都做一遍特征匹配
这需要 C_N^2 的复杂度

2. 随机抽取历史数据并进行回环检测

随着 N 的增长，抽取到回环的概率会显著减小

3. 避免进行盲目的匹配的方法

基于里程计的几何关系，里程计预计闭环的时候才进行检测，但无法产生最有用的闭环检测信息

基于图像外观，是现在常用的方法，如何计算图像间的相似性？

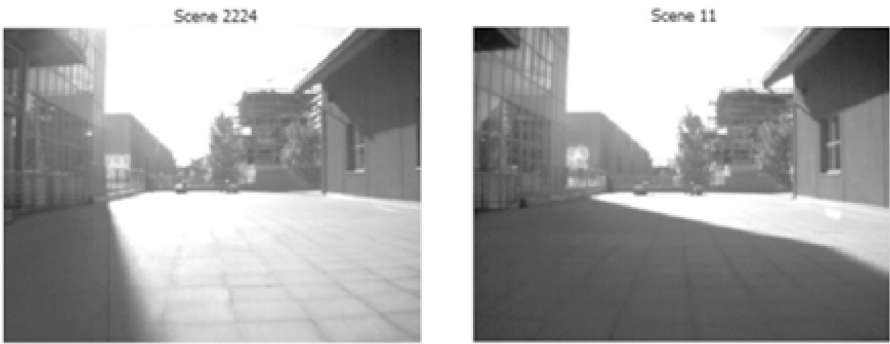
准确率(Precision)和召回率(Recall)

表 12-1 回环检测的结果分类

算法 \ 事实	是回环	不是回环
是回环	真阳性 (True Positive)	假阳性 (False Positive)
不是回环	假阴性 (False Negative)	真阴性 (True Negative)



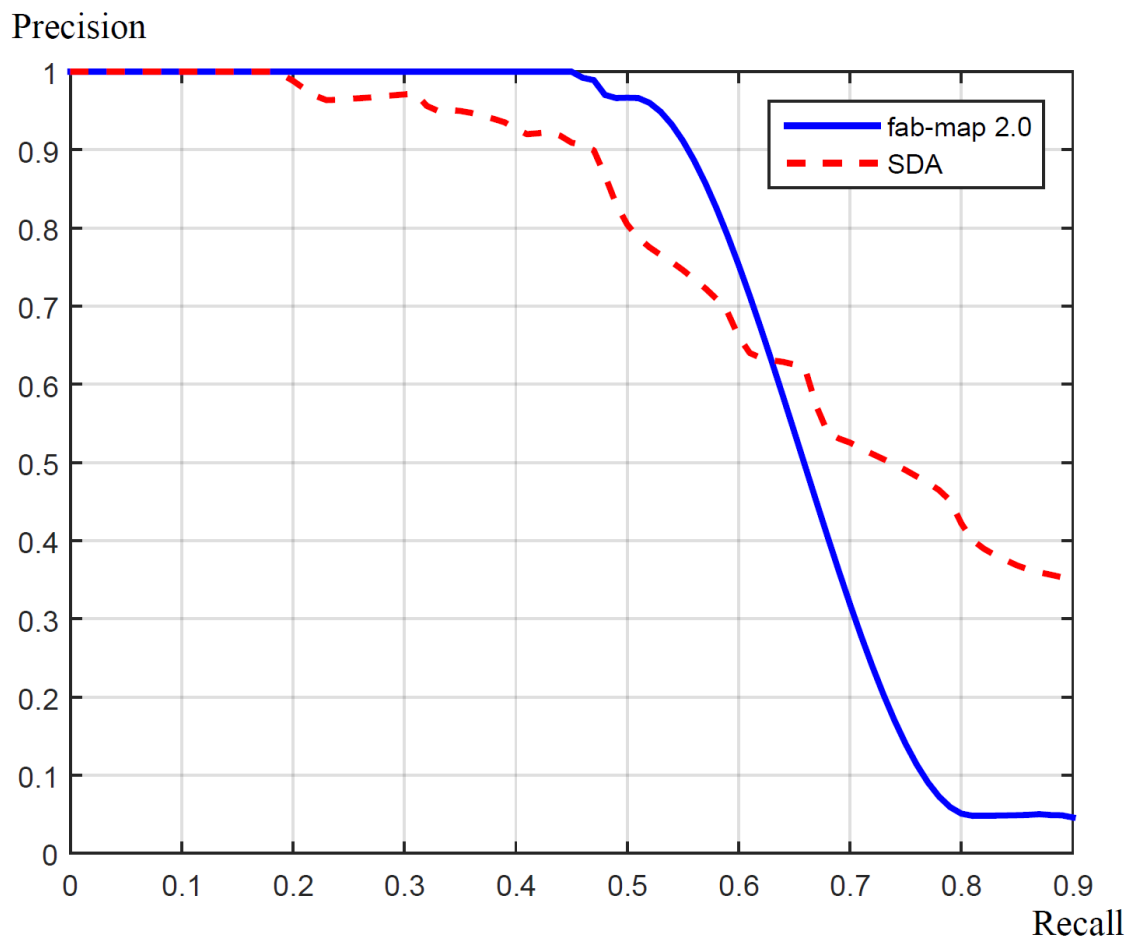
False Positive



False Negative

准确率 (Precision) 和召回率 (Recall)

$$\text{Precision} = TP / (TP + FP), \quad \text{Recall} = TP / (TP + FN)$$



准确率是检出的
回环都是正确的
比率

召回率是真正的
回环不被漏检的
比率

SLAM的闭环检测
要求高准确率,
不要求高召回率

基于图像特征的回环检测

1. 回环检测可以采用的方案有哪些呢？

1. 基于特征点匹配？

- 例如提取ORB特征点，并进行特征点匹配，这种方法由于特征点提取、特征点描述和特征点匹配都比较消耗时间，所以效率不高。

• 基于光流法或直接法？

- 光流法和直接法都比较依赖于较好的相机姿态初值，适合于在连续运动过程中，待匹配目标点在投影点邻域附近的情况，可以有效的进行邻域搜索。在大范围寻找回环时，不是很适用。

• 基于词袋模型的回环检测？

- 将图像上的特征点用词向量来描述，通过词向量匹配方法，先确定两帧图像之间的相似关系

基于词袋模型进行回环检测

Galvez-López D, Tardos J D. **Bags of Binary Words for Fast Place Recognition in Image Sequences**[J]. IEEE Transactions on Robotics, 2012, 28(5):1188-1197.

1.提取特征

2.构建字典（所有单词的集合）

$$D=(\omega_1,\omega_2,\omega_3\cdots\omega_{n-1},\omega_n)$$

3.确定一帧中具有哪些单词，形成词袋向量
(1表示具有该单词，0表示没有)

$$F=1\cdot\omega_1+0\cdot\omega_2+0\cdot\omega_3+\cdots+1\cdot\omega_{n-1}+0\cdot\omega_n$$

4.比较两帧描述向量的差异。

词袋模型Bag-of-Words (BoW)

在自然语言处理中用于识别相似的文本

文本中出现的单词：人、车、狗

词典：离线构造的所有可能出现的单词的集合，设存储 N 个单词。

词频向量：按字典顺序用 N 维，0-1向量描述的某一文本中单词出现的情况。

向量比较：其中范数取 L_1 范数，即各元素绝对值之和：

$$s(a, b) = 1 - \frac{1}{W} \|a - b\|_1$$

在SLAM中用于回环检测

图像中出现的特征点：brief descriptor

词典：离线构造的各类图像中可能出现的特征描述子所构成的词典

词频向量：按字典顺序，用一个 N 维，0-1向量描述某一帧图像中，字典中单词出现的情况

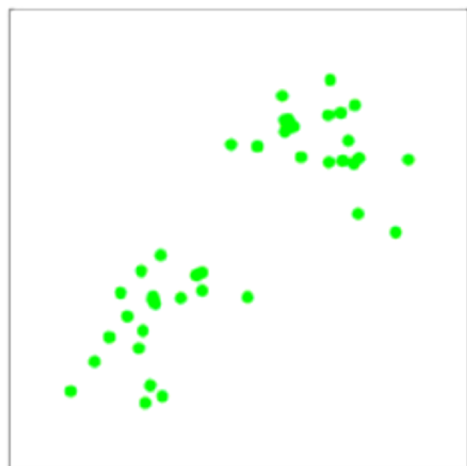
向量比较：仍然是计算 L_1 范数，但是引入了TF-IDF权重

基于词袋模型进行回环检测的流程

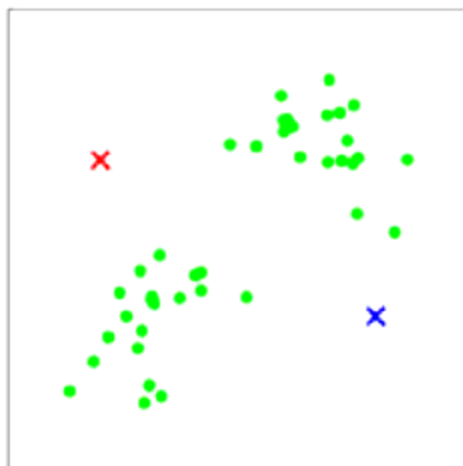


1. 词典的构造

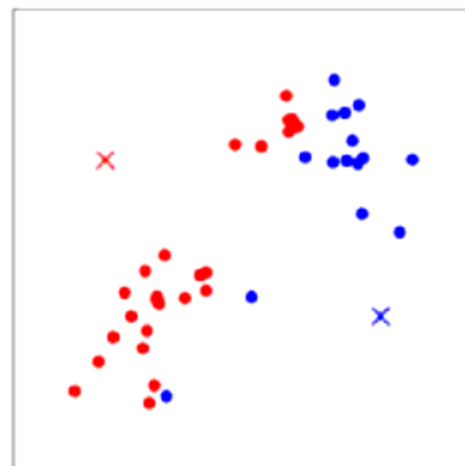
- 字典生成问题类似于一个**聚类**（Clustering）问题，无监督机器学习（Unsupervised ML）自动产生聚类的方法。
- 假设我们对大量的图像提取了特征点，比如说有 N 个。现在，我们想找一个有 k 个单词的字典，每个单词可以看作局部相邻特征点的集合。
- K-means 是一个非常简单有效的方法，当我们将有 N 个数据，想要归成 $k < N$ 个类
 1. 随机选取 k 个中心点： $c_1, \dots, c_k$;
 2. 对每一个样本，计算与每个中心点之间的距离，取最小的作为它的归类;
 3. 重新计算每个类的中心点。
 4. 如果每个中心点都变化很小，则算法收敛，退出；否则返回 1。



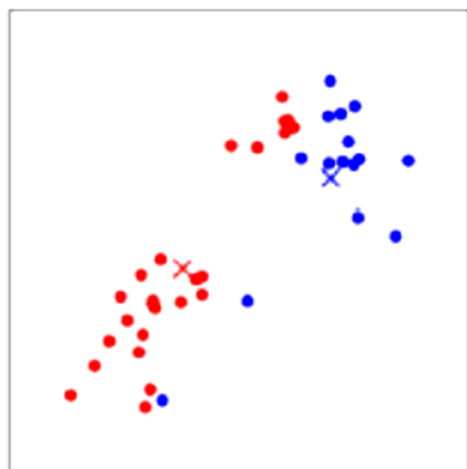
(a)



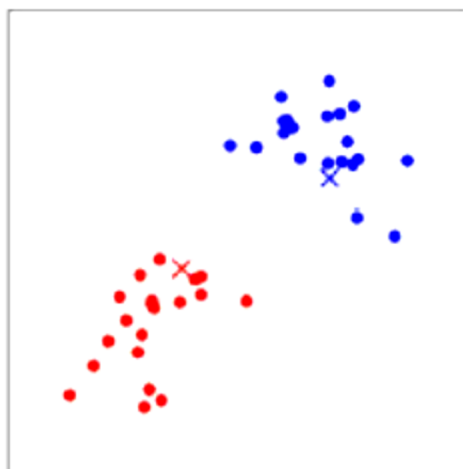
(b)



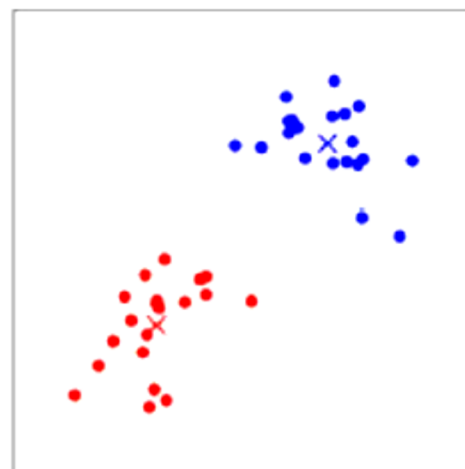
(c)



(d)



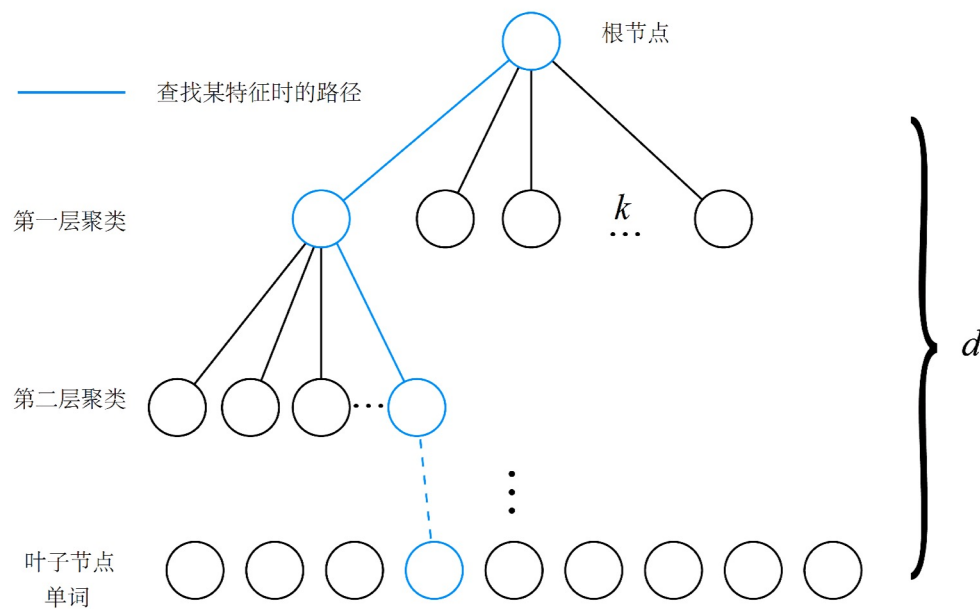
(e)



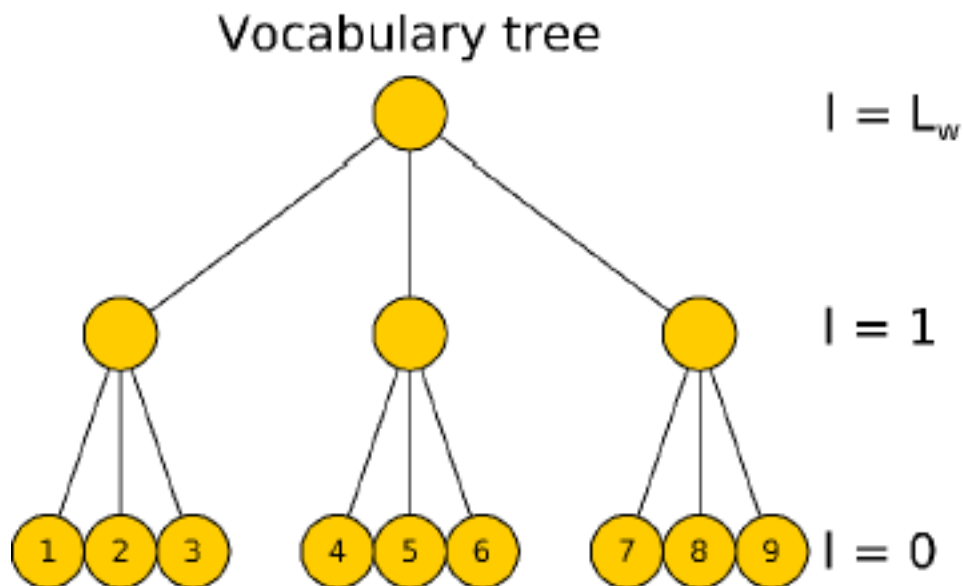
(f)

1. 词典的构造，层次聚类法

1. 在根节点，用 k-means 把所有样本聚成 k 类（实际中为保证聚类均匀性会使用 k-means++）。这样得到了第一层。
2. 对第一层的每个节点，把属于该节点的样本再聚成 k 类，得到下一层。
3. 依此类推，最后得到叶子层。叶子层即为所谓的 Words。



1. 词典的构造



Vocabulary branch factor: k

Vocabulary depth levels: L_w

- 词典是基于训练图像离线构造的。
- 每个单词是一些特征点 descriptor 的 cluster head
- 设最终词典中有 N 个单词, $N = k^{L_w}$
- 对于待查询图像中的 descriptor, 通过 vocabulary tree 进行查找时, 仅需要进行 L_w 次比较, 就可以找到最接近的单词。

字典构造和查找的开源库

DBOW

DBow库是一个开源C++库，用于索引图像并将图像转换为词袋表示。它实现了一个分层树，用于近似图像特征空间中的最近邻并创建可视词汇表。DBow还实现了一个基于逆序文件结构的图像数据库，用于索引图像和快速查询。

源码地址：<https://github.com/dorian3d/DBow>

DBOW2

DBow2实现了具有正序和逆序指向索引图片的的图像数据库，可以实现快速查询和特征比较。在几个真实数据集上进行了测试，3毫秒可以将图像的简要特征转换为词袋向量，5毫秒可以在数据库中查找图像匹配超过19000张图片。

源码地址：<https://github.com/dorian3d/DBow2>

DBow3

DBow3是DBow2库的改进版本

源码地址：<https://github.com/rmsalinas/DBow3>

FBOW

Fast Bag of Words是DBow2 / DBow3库的极端优化版本，比DBOW2快80倍。

源码地址：<https://github.com/rmsalinas/fbow>

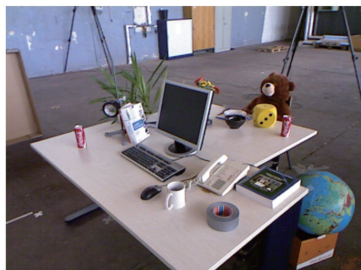
相关介绍：<https://www.cnblogs.com/CV-life/p/11420320.html>

实践：创建字典

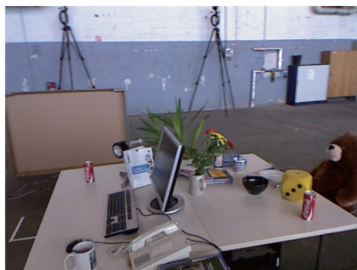
如何创建和使用ORB字典



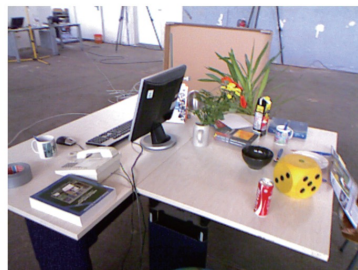
1



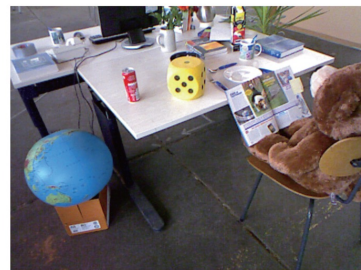
2



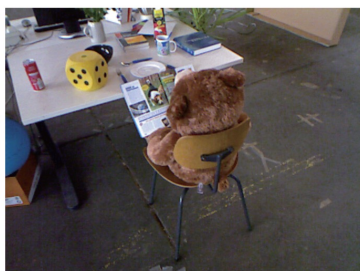
3



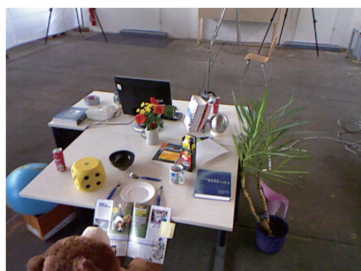
4



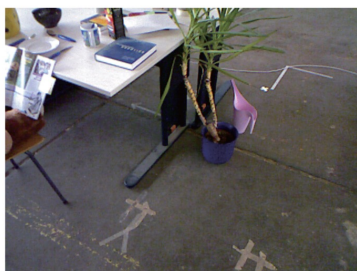
5



6



7



8



9



10

10幅不同位置采集到的图像

第1 张图像与最后一张图像采自同一个地方

使用开源库DBoW3 创建字典

```
// detect ORB features
cout<<"detecting ORB features ... "<<endl;
Ptr< Feature2D > detector = ORB::create();
vector<Mat> descriptors;
for ( Mat& image:images )
{
    vector<KeyPoint> keypoints;
    Mat descriptor;
    detector->detectAndCompute( image, Mat(), keypoints, descriptor );
    descriptors.push_back( descriptor );
}
```

```
// create vocabulary
cout<<"creating vocabulary ... "<<endl;
DBoW3::Vocabulary vocab;
vocab.create( descriptors );
cout<<"vocabulary info: "<<vocab<<endl;
vocab.save( "vocabulary.yml.gz" );
cout<<"done"<<endl;
```

使用开源库DBoW3 创建字典

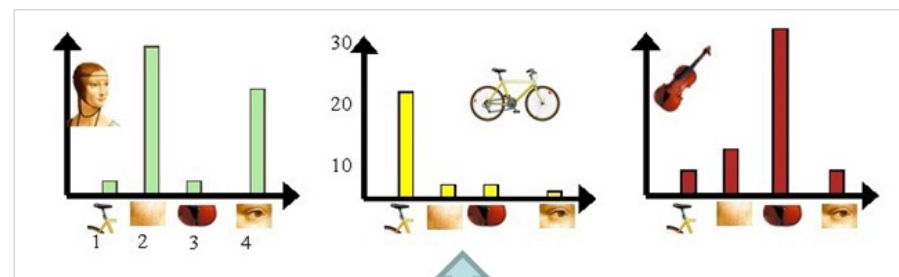
字典中存储的一棵树， k=10, L=5, 树中存储node, parent, descriptor

```
%YAML:1.0
vocabulary:
  k: 10
  L: 5
  scoringType: 0
  weightingType: 0
  nodes:
    - { nodeId:1, parentId:0, weight:0.,
      descriptor:"0 32 27 159 99 205 118 146 238 119 125 72 94 230 121 238 182 228 126 239 23 237 197 184 37 239 54 110 170 128 47 174 121 221 " }
    - { nodeId:2, parentId:0, weight:0.,
      descriptor:"0 32 0 141 27 249 52 207 99 119 116 1 110 0 183 19 32 96 120 58 6 8 59 184 113 63 229 175 50 64 48 136 224 168 " }
    - { nodeId:3, parentId:0, weight:0.,
      descriptor:"0 32 88 89 255 245 34 66 127 48 164 206 214 162 253 55 129 115 102 245 125 234 233 209 177 108 243 235 238 128 36 176 37 121 " }
    - { nodeId:4, parentId:0, weight:0.,
      descriptor:"0 32 88 184 190 176 169 236 17 7 171 31 166 25 190 191 66 91 167 183 253 16 26 77 146 193 219 243 94 123 128 64 135 91 " }
    - { nodeId:5, parentId:0, weight:0.,
      descriptor:"0 32 245 127 237 101 172 15 207 184 80 250 150 238 211 109 217 107 94 196 124 255 201 246 253 136 236 139 229 180 109 251 82 113 " }
    - { nodeId:6, parentId:0, weight:0.,
      descriptor:"0 32 2 188 184 186 253 237 145 207 158 61 136 25 188 255 104 72 201 27 251 17 18 75 218 243 205 151 31 127 144 72 133 234 " }
    - { nodeId:7, parentId:0, weight:0.,
      descriptor:"0 32 56 249 26 254 93 205 86 230 143 5 236 9 175 219 98 144 200 251 199 66 78 43 90 181 251 213 52 59 82 26 206 138 " }
    - { nodeId:8, parentId:0, weight:0.,
      descriptor:"0 32 77 121 91 101 52 14 111 48 108 168 126 76 215 19 145 48 54 108 124 110 193 176 241 12 229 189 33 0 72 179 67 33 " }
    - { nodeId:9, parentId:0, weight:0.,
      descriptor:"0 32 61 69 107 239 116 27 111 248 87 224 255 70 211 97 185 112 122 204 30 238 237 178 109 159 246 239 165 148 125 187 112 161 " }
    - { nodeId:10, parentId:0, weight:0.,
      descriptor:"0 32 2 157 34 153 255 213 184 199 159 69 88 183 172 222 54 196 239 59 147 149 150 186 50 247 63 84 154 235 151 12 189 223 " }
    - { nodeId:5376, parentId:10, weight:0.,
      descriptor:"0 32 10 150 234 153 246 183 185 91 31 95 87 183 140 142 190 76 255 27 151 149 182 156 37 215 78 95 186 199 151 140 181 215 " }
    - { nodeId:5377, parentId:10, weight:0.,
      descriptor:"0 32 35 188 98 154 221 245 178 207 223 69 124 15 40 252 118 164 221 43 147 21 150 58 254 247 63 93 159 107 211 78 253 222 " }
    - { nodeId:5378, parentId:10, weight:0.,
      descriptor:"0 32 10 185 50 156 123 208 184 71 175 69 124 163 44 158 38 212 230 59 215 129 22 11 178 245 23 117 154 203 19 4 175 223 " }
    - { nodeId:5379, parentId:10, weight:0.,
      descriptor:"0 32 2 141 174 191 223 196 188 231 158 87 90 151 172 238 54 65 231 59 187 133 118 217 18 255 173 54 222 195 148 76 189 222 " }
    - { nodeId:5380, parentId:10, weight:0.,
      descriptor:"0 32 14 122 162 154 239 245 184 192 159 255 148 167 172 222 119 206 239 121 251 215 134 71 170 240 31 85 223 175 151 125 61 215 " }
```

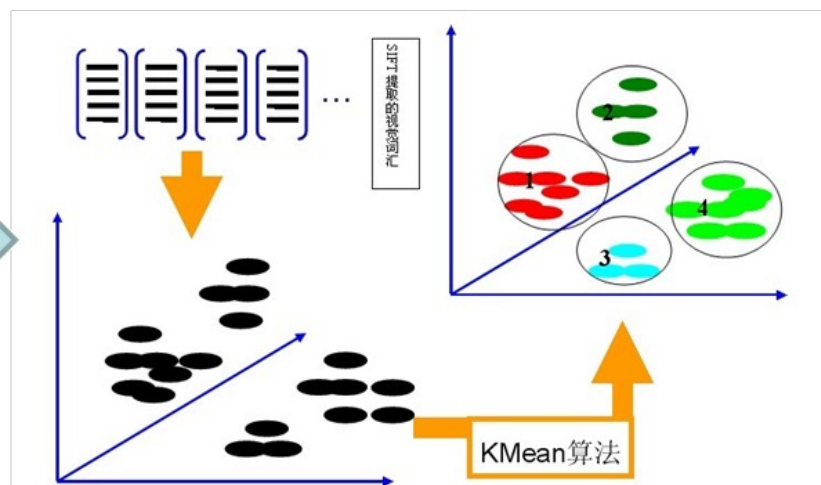
2. 基于词典对查询图像进行编码

在离线建立词典之后，在在线回环检测过程中，从当前查询图像中提取特征点，并利用词典树查找匹配的单词，从而确定当前查询图像的词向量表示。

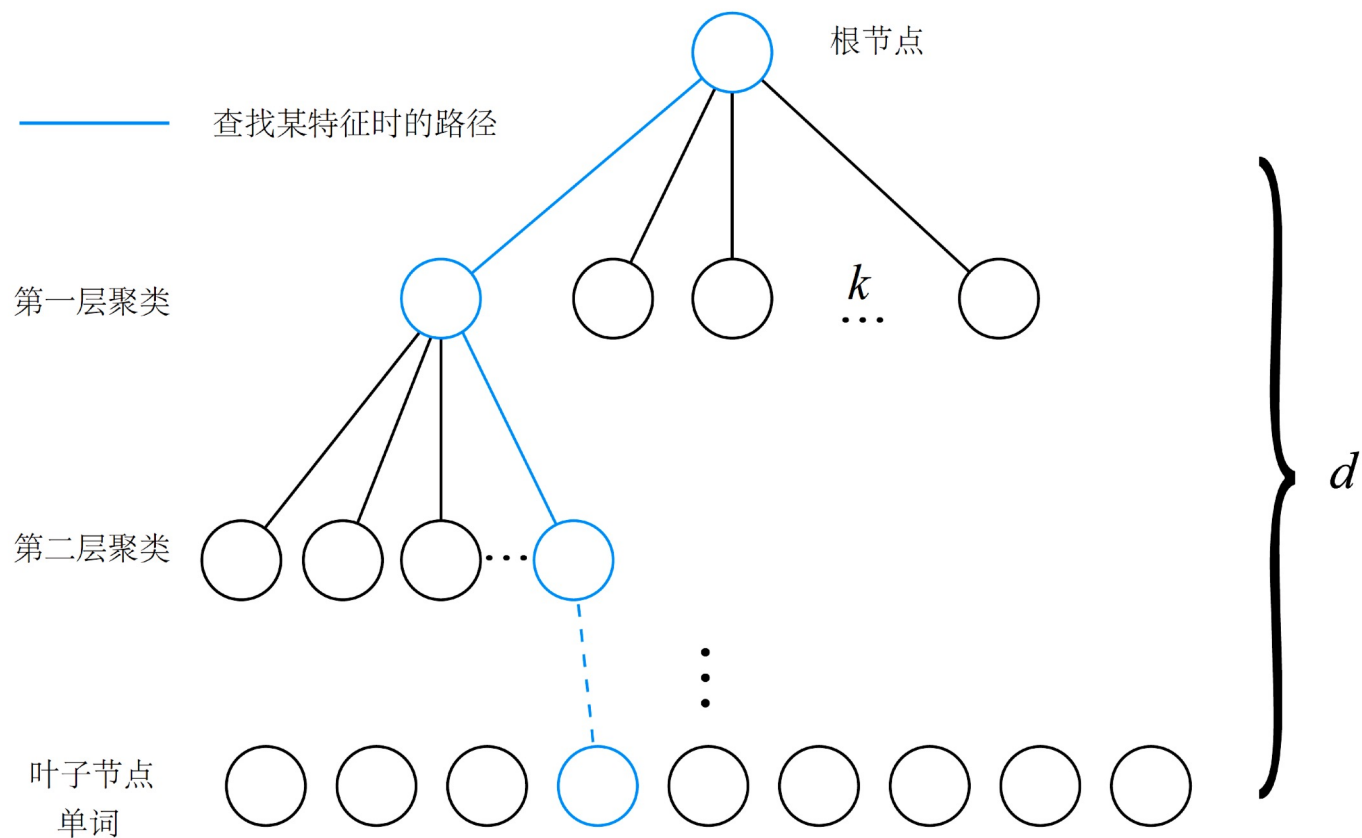
查询图像编码过程



词典建立过程



2. 图像—>BoW向量流程



每个图像的包含多个特征，对每个特征查询其对应的word，最后用长度为 N 的词频向量描述这个图像

$$F = 1 \cdot \omega_1 + 0 \cdot \omega_2 + 0 \cdot \omega_3 + \cdots + 1 \cdot \omega_{n-1} + 0 \cdot \omega_n$$

图像编码为 $[1 \ 0 \ 0 \ \cdots \ 1 \ 0]$

2. 图像—>BoW向量流程

1、输入一张图像，进行特征检测与特征描述，特征点数量最多不超过约定值(300)。

2、将每个特征通过树形结构的字典，得到BoW向量 $\mathbf{v}$ ，该向量的维度为叶节点（words）数量。

其中向量 $\mathbf{v}$ 的数据单元结构为map—>(index , value)即(word index, weight)

word index 是匹配的word在词典中的index，weight 通过 TF-IDF进行计算。

$$w_t^i = \text{tf}(i, I_t) \times \text{idf}(i), \quad w_t^i = w_t^i / \sum w_t^i$$

$$\text{tf}(i, I_t) = \frac{n_{iI_t}}{n_{I_t}}, \quad \text{在生成Bow向量时产生的权重分量} \quad \text{由查询图片确定}$$

$$\text{idf}(i) = \log \frac{N}{n_i}, \quad \text{生成字典时产生的权重分量} \quad \text{由字典确定}$$

2.图像—>BoW向量流程，权重的计算

$$\text{tf}(i, I_t) = \frac{n_{iI_t}}{n_{I_t}}, \text{ 在生成Bow向量时产生的权重分量}$$

分子为当前图片中出现单词*i*的个数，分母为总共包含的单词个数，这是在线图像中计算得到的。

$$\text{idf}(i) = \log \frac{N}{n_i}, \text{ 生成字典时产生的权重分量}$$

这是词典的属性，已经离线生成，不会发生改变，分子为生成词典时所包含的训练图片数量，分母为出现单词*i*的图片个数。

这个比重分量主要为了凸显差异型，认为在训练时经常出现的单词差异性较小，权重赋予的小一些。

$$w_t^i = \text{tf}(i, I_t) \times \text{idf}(i), \quad w_t^i = w_t^i / \sum w_t^i$$

最后对图像中出现的单词计算权重，并对所有出现的单词权重归一化。

2.图像—>BoW向量流程， 相似度计算

查询在线检测的图像与图像数据库中哪个图像接近的时候， 需要计算图像词向量相似度。

$$L_1 - \text{score } s(v_1, v_2) = 1 - \frac{1}{2} \left| \frac{v_1}{|v_1|} - \frac{v_2}{|v_2|} \right|$$

1-范数是求向量元素的绝对值之和， 两个词向量越接近， 相似度Score越接近于1

实际中不可能存一个100万维的BoW向量， 向量是稀疏的， 程序中只存下出现某个单词的索引及权重

具体计算过程逐个找到index 相同的word， 计算相似度， 然后累加。

$$s(v_1, v_2) = \sum (|v_1 - v_2| - |v_1| - |v_2|)$$

逐个找到index相同的单词， 累加相似度
越相似数值越小。

$$L_1 - \text{score } s(v_1, v_2) = -\frac{1}{2} s(v_1, v_2)$$

取反乘以1/2， 越相似数值越大， 范围0-1

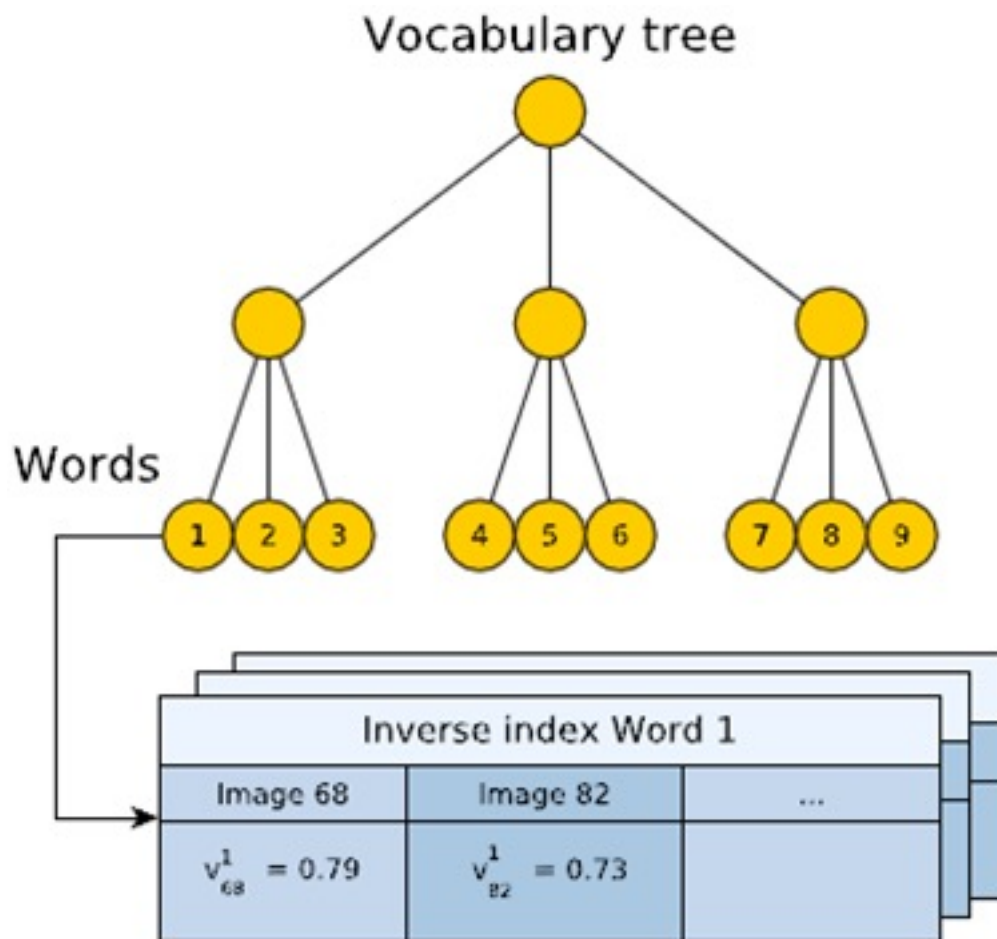
3. 基于词典查找匹配图像

在SLAM过程中，累计的关键帧会越来越多，需要同所有已知图像匹配么？

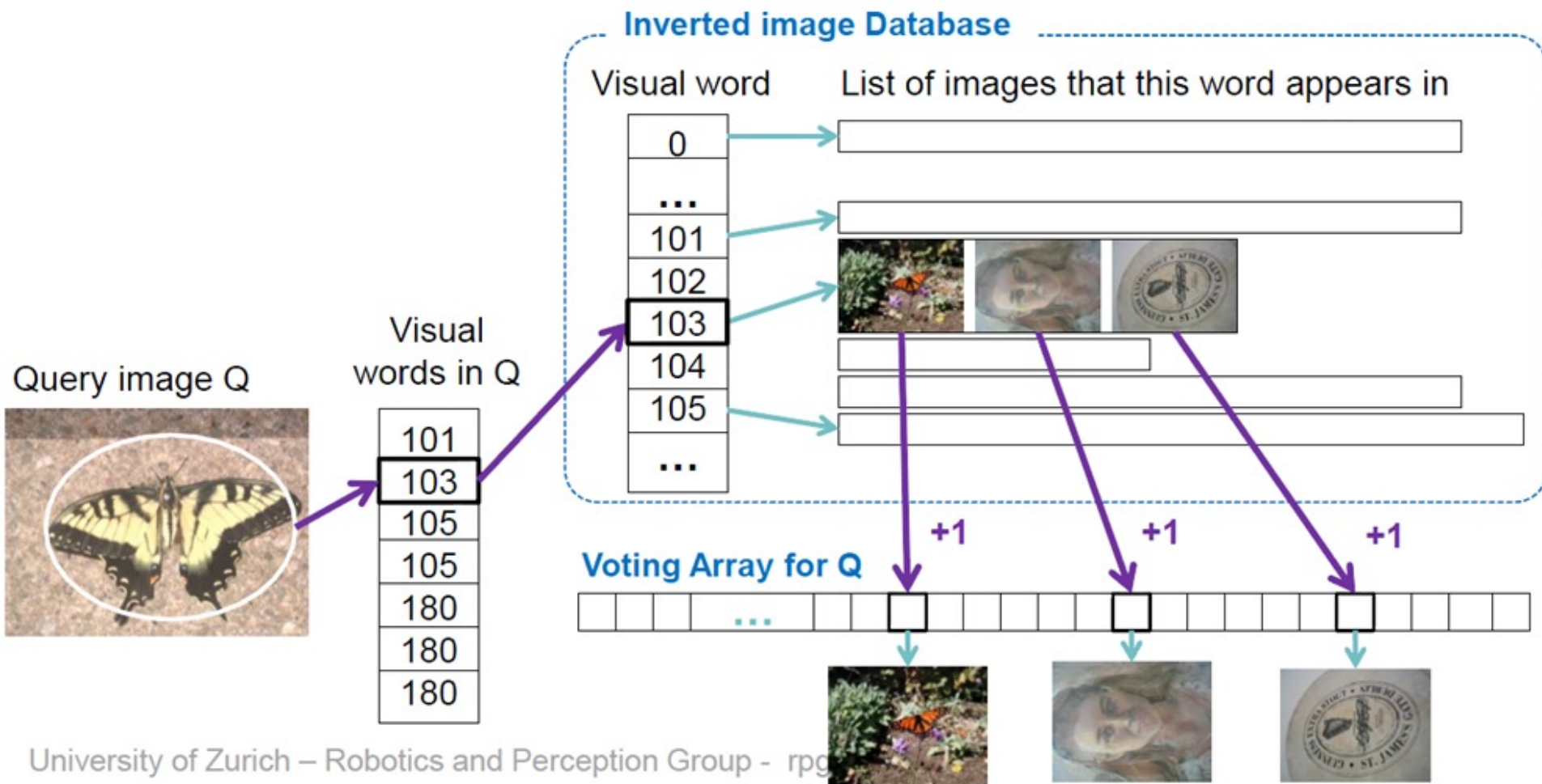
边收集图像边建立Image Database, 通过Inverse Index **为每个word存储其出现的图像列表和这个word在该图像中的权重。**

逆序索引单词涵盖所有出现的单词，每一个单词指向包含它的一系列图像

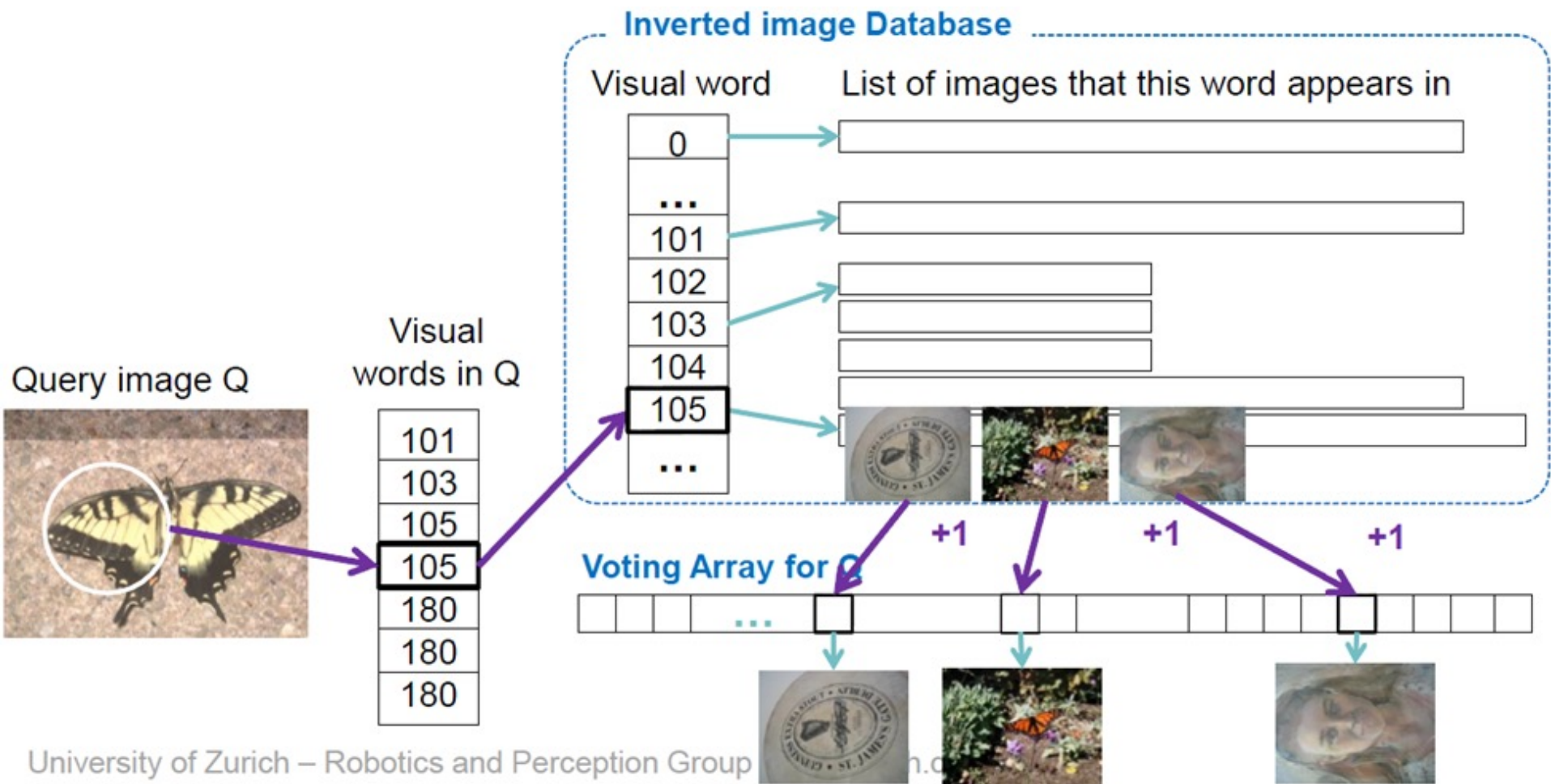
采用投票机制选取候选者，加速匹配搜索



3. 基于词典查找匹配图像的加速方法



3. 基于词典查找匹配图像的加速方法



基于投票结果对可能匹配的图像进行排序，这样只有包含查询图像中word的相关图像会被选出，不会涉及数据库中的所有图像。

4. 基于词典查找匹配图像，减少误报方法

计算匹配图像的相似度同与前一帧图像的相似度的比值，得到归一化得分函数，变绝对相似度为相对相似度。

$$\eta(v_t, v_{t_j}) = \frac{s(v_t, v_{t_j})}{s(v_t, v_{t-\Delta t})}$$

1. 防止因分母过小，而引起的得分过大，需要对分母增加条件(有特征点数要求或者得分要求)
2. 需要对最后得分设立阈值，过滤得分较少的候选图像，保留符合要求的候选图像进入组匹配进行校验
3. 例如设阈值为3.

4. 基于词典查找匹配图像，减少误报方法

利用当图像 $I_t, I_{t'}$ 表示了一个真正的回环，则 I_t 同样也和 $I_{t' \pm \Delta t}, I_{t' \pm 2\Delta t}$ 有着较高的相似性，用一段滑窗匹配降低匹配的误报率，并提高准确率。

定义一个块得分函数

$$H(v_t, V_{T_i}) = \sum_{j=n_i}^{m_i} \eta(v_t, v_{t_j})$$

其中 T_i 表示候选图像所在集合从 n_i 到 m_i

4. 基于词典查找匹配图像，时间一致性校验 减少误报

在经过组匹配之后，得到了最佳组匹配 V_T ，此时需要检验该组匹配的时间一致性。

在得到组匹配之后，这组匹配图像与当前帧的前 k 帧图像得到的组匹配 $\langle v_{t-1}, V_{T1} \rangle$ ， $\langle v_{t-2}, V_{T2} \rangle$ ， $\langle v_{t-3}, V_{T3} \rangle$ 应该比较相似。

如果经过时间一致性校验之后，存在上述特点，则说明这个匹配是可靠的。在上述组匹配中，只保留一组得分最高的，进入结构一致性校验。

5. 基于词典查找匹配图像，结构一致性校验

词袋是一个无序的模型，会带来False Negative 匹配，例如，小王成绩不如小张 和小张成绩不如小王，意思相反，但在词袋模型上两者完全一样。

词袋模型匹配过程中只考虑了特征点的出现，没有考虑特征点的空间位置，所以最后一步，基于特征点的空间位置进行结构性检验，消除进一步消除FN匹配。

对于每对匹配的image candidate，需要通过RANSAC找至少12对距离最近的特征点，计算基础矩阵，进行结构一致性验证。如果不同组合算出来的基础矩阵一致性较好，说明确实找到了结构一致的回环。

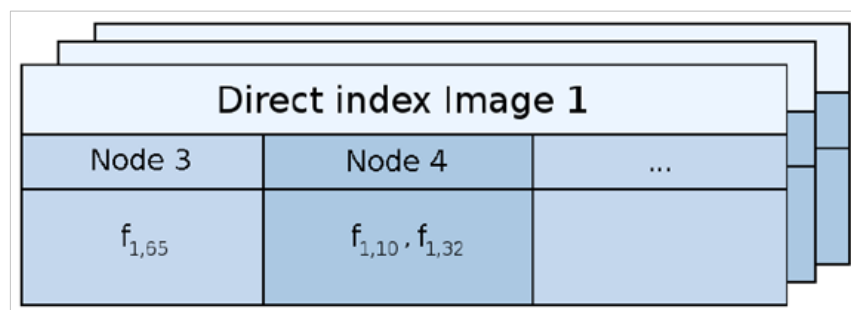
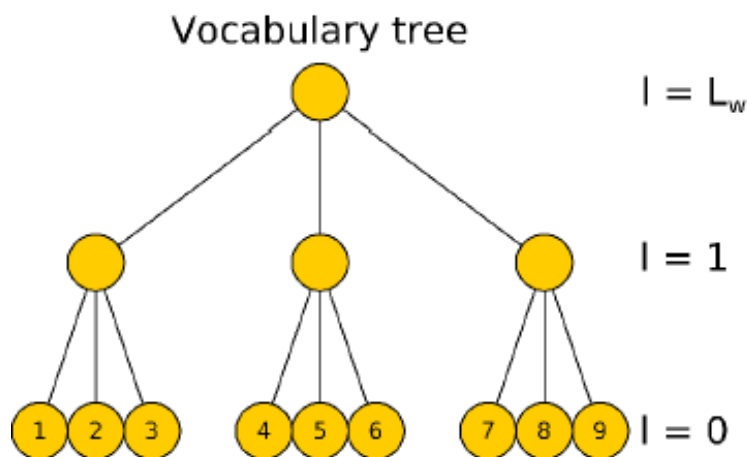
但找两幅图像中距离最近的特征点可以用特征点之间的暴力匹配，计算复杂度为 $O(n^2)$ ， n 为图像中特征点个数，还是比较耗时的。

5. 基于词典查找匹配图像， 结构一致性校验

结构一致性检验的加速方法

在建立Image Database的时候，对每个图像维护一个direct index，记录每个feature存在了词典树的哪个node下面。

假设图像 I_i 和 I_j 得分最高，在两幅图像特征匹配时，只需要比较 direct index 中属于同一层 node 的图像特征，node 指字典树的一层，如果是叶子节点层，那么选择是同一个 word 的特征做匹配。



例如取 $l=1$ ，则只找 I_i 这一层节点包含的 feature 对应于 I_j 同一层节点所包含的 feature

BoW回环检测整体算法流程

离线工作

用大量的训练图像提取特征点，进行特征层次聚类，离线构造BoW字典

词典

获得图像
提取特征点

在词典树上查找图
像的词向量表示

产生带权重的词向量

将词向量和Image存
入Image Database

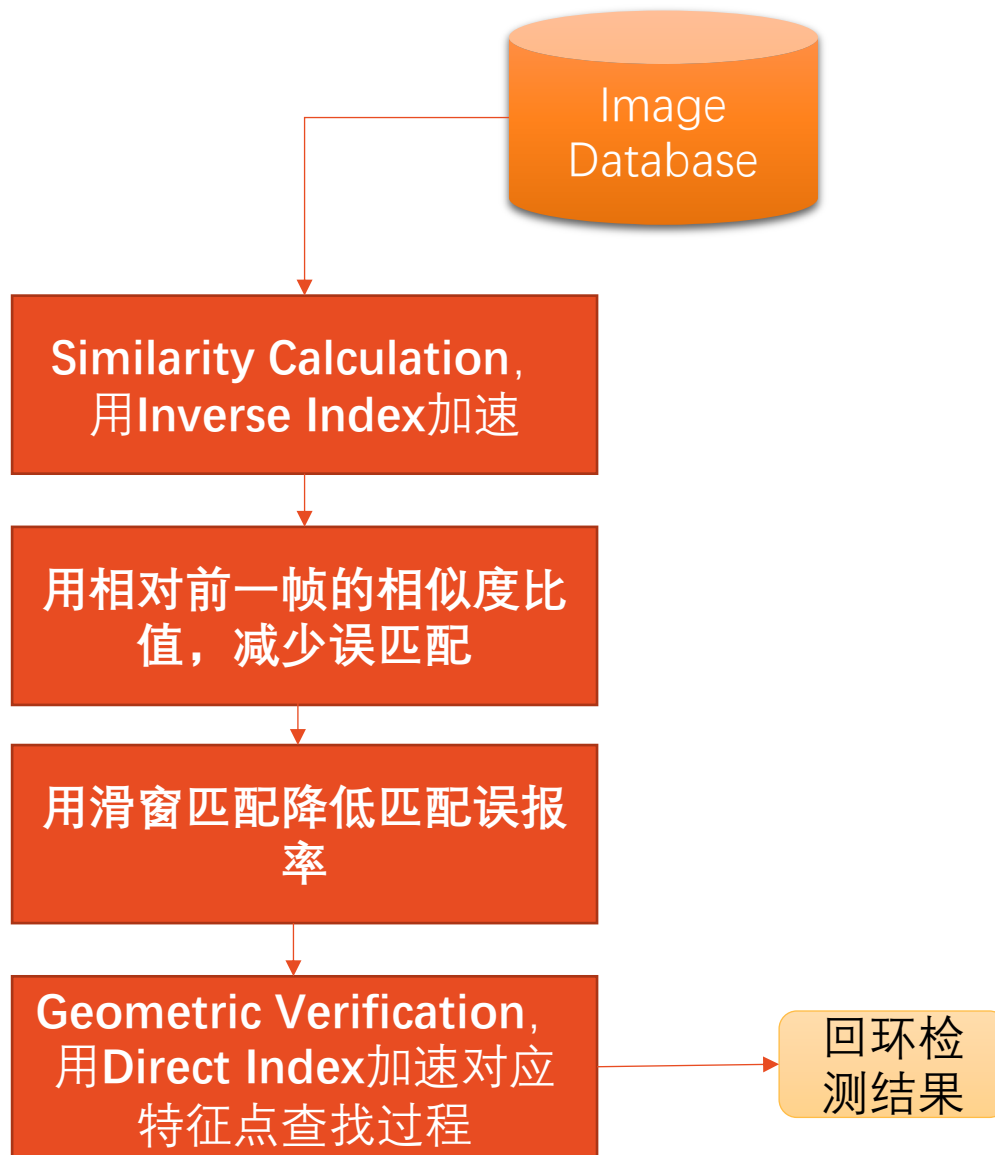
Image
Database

更新Image Database 中
单词的 Inverse Index

更新Image Database中
单词的 Direct Index

图像获得词向量描述部分

BoW回环检测整体算法流程



在线的回环检测部分

实践：相似度的计算

```
cout << "reading database" << endl;
DBow3::Vocabulary vocab("../vocabulary.yml.gz");
// DBow3::Vocabulary vocab("../vocab_larger.yml.gz");
if (vocab.empty()) {
    cerr << "Vocabulary does not exist." << endl;
    return 1;
}
```

```
// detect ORB features
cout << "detecting ORB features ... " << endl;
Ptr<Feature2D> detector = ORB::create();
vector<Mat> descriptors;
for (Mat &image:images) {
    vector<KeyPoint> keypoints;
    Mat descriptor;
    detector->detectAndCompute(image, Mat(), keypoints, descriptor);
    descriptors.push_back(descriptor);
}
```

直接比较图片的“words”

```

cout << "comparing images with images " << endl;
for (int i = 0; i < images.size(); i++) {
    DBoW3::BowVector v1;
    vocab.transform(descriptors[i], v1);
    for (int j = i; j < images.size(); j++) {
        DBoW3::BowVector v2;
        vocab.transform(descriptors[j], v2);
        double score = vocab.score(v1, v2);
        cout << "image " << i << " vs image " << j << " : " << score << endl;
    }
    cout << endl;
}

```

// or with database

```

cout << "comparing images with database " << endl;
DBoW3::Database db(vocab, false, 0);
for (int i = 0; i < descriptors.size(); i++)
    db.add(descriptors[i]);
cout << "database info: " << db << endl;
for (int i = 0; i < descriptors.size(); i++) {
    DBoW3::QueryResults ret;
    db.query(descriptors[i], ret, 4); // max result=4
    cout << "searching for image " << i << " returns " << ret << endl << endl;
}

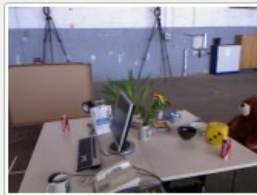
```



1.png



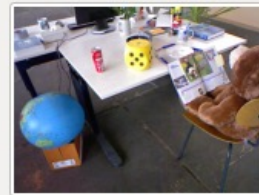
2.png



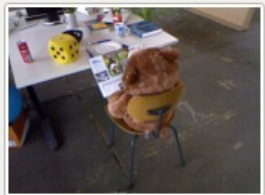
3.png



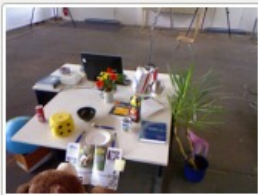
4.png



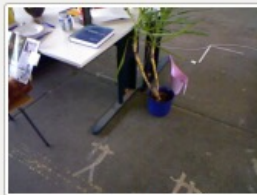
5.png



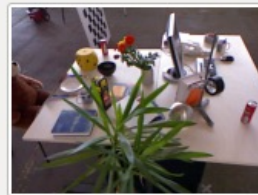
6.png



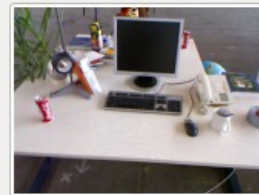
7.png



8.png



9.png



10.png

目测1-10是回环

```
comparing images with images
image 0 vs image 0 : 1
image 0 vs image 1 : 0.0356429
image 0 vs image 2 : 0.027241
image 0 vs image 3 : 0.0257926
image 0 vs image 4 : 0.03394
image 0 vs image 5 : 0.0211594
image 0 vs image 6 : 0.03589
image 0 vs image 7 : 0.0291903
image 0 vs image 8 : 0.0367468
image 0 vs image 9 : 0.0291648
```

检测结果并不对

```
comparing images with database
database info: Database: Entries = 10, Using direct index = no.
1-norm, Number of words = 99566
searching for image 0 returns 4 results:
<EntryId: 0, Score: 1>
<EntryId: 8, Score: 0.0367468>
<EntryId: 6, Score: 0.03589>
<EntryId: 1, Score: 0.0356429>
```

回环检测小结

回环检测是在图优化中建立“超级边”的关键

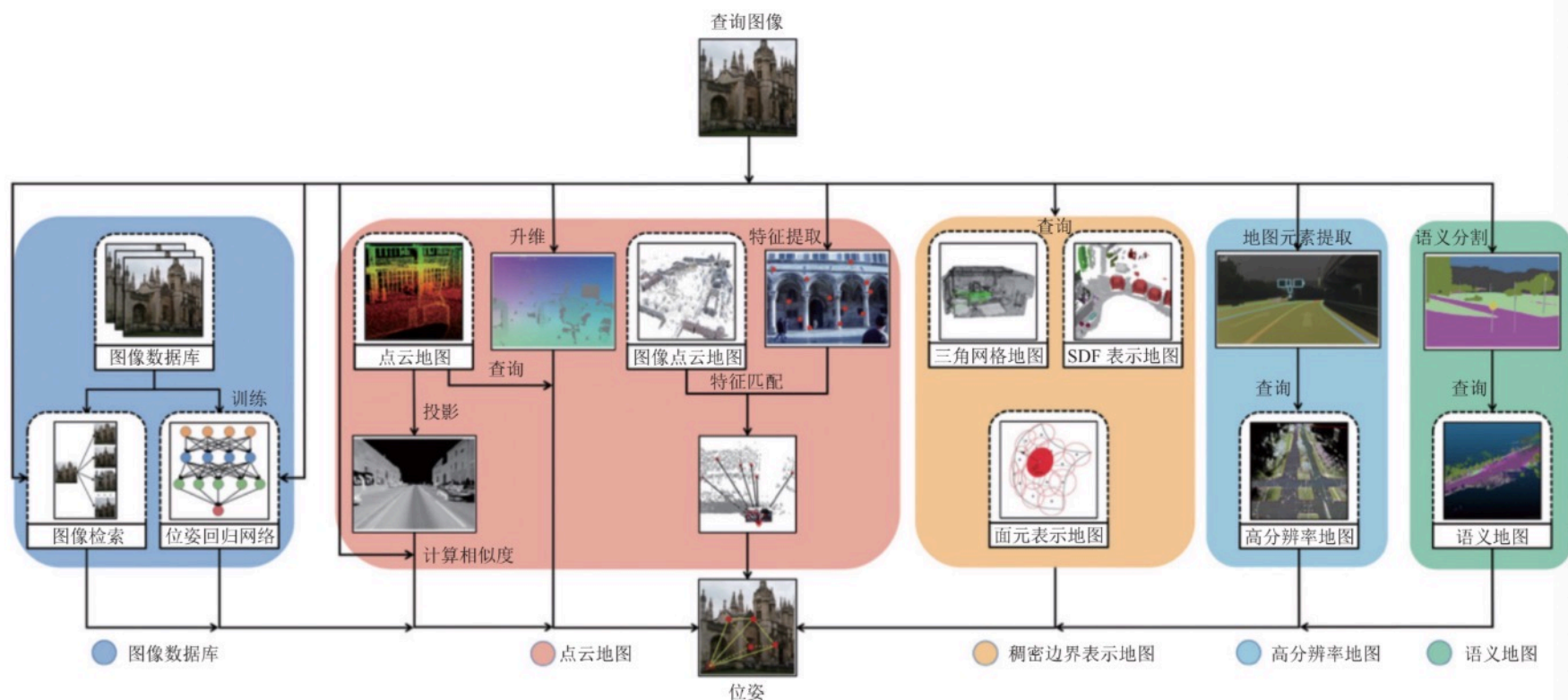
现有基于词袋模型的方法是容易产生误匹配

主要通过多种一致性检查消除误匹配，但可能导致一些回环找不到。

基于语义和机器学习的回环检测可以加强这一部分

大规模地图的重定位和回环检测还很值得研究

其它基于深度学习的视觉重定位方法



(1) 基于**图像数据库的视觉重定位**: 由于地图数据与查询图像属于相同模态, 主要重定位方法分为:

1) 使用图像检索方法直接在图像数据库中查询[43]。

2) 使用深度学习方法在位姿回归网络中直接进行位姿回归计算[44]。

(2) 基于**点云数据库的视觉重定位**: 点云数据库与查询图像处于不同模态, 无法直接比较, 目前主流方法分为:

1) 基于**点云投影的方法**先将点云投影为 **2D 图像**, 再借助图像相似度进行查询[45]。

2) 基于**场景升维方法**先将查询图像升到三维, 再在点云地图数据库中进行查询[46]。

3) 基于**特征点匹配的方法**利用

SfM (structure from motion)[47–49]构建带有图像特征**的点云地图数据库** 再基于图像特征匹配关系解算位姿[50]。

视觉重定位方法综述

(3) 稠密边界表示地图的视觉重定位:

- 主要包括三角网格地图、**SDF** 表示地图、面元表示地图。
- 稠密边界地图中进行查询的方法主要利用场景的几何结构和丰富的纹理信息与查询图像建立数据关联[51]。

(4) 高分辨率地图数据库的视觉重定位:

- 高分辨率地图构建时主要进行地图元素提取, 以矢量形式保存地图元素。
- 查询时先从查询图像中提取出同种地图元素之后, 再进行匹配与查询[52]。

(5) 语义地图数据库的视觉重定位:

- 语义数据库中包含了地图元素的语义信息, 可以利用查询图像的语义信息作为全局特征进行查询[53]也可以利用语义标签之间的对应关系进行查询[54]。
- 语义信息还可以用于指导算法选择更鲁棒的特征进行查询[55]。